

Technical Documentation

Odoo POS ↔ Oracle NetSuite Integration

Project Name: Odoo Point of Sale to NetSuite ERP Integration **Date:** May 13, 2026 **Classification:** Technical Implementation Specification

Document Purpose

This technical documentation provides comprehensive implementation details for the Odoo POS to NetSuite ERP integration solution. It serves as the authoritative reference for developers, system administrators, and technical stakeholders involved in deployment, maintenance, and enhancement of the integration system.

Table of Contents

- 1. [Executive Summary](#)
- 2. [System Architecture](#)
- 3. [Odoo Implementation Details](#)
- 4. [NetSuite Implementation Details](#)
- 5. [API Specifications](#)
- 6. [Data Flow and Process Logic](#)
- 7. [Configuration Management](#)
- 8. [Security and Authentication](#)
- 9. [Error Handling and Logging](#)
- 10. [Deployment Guide](#)
- 11. [Testing Strategy](#)
- 12. [Appendices](#)

1. Executive Summary

1.1 Project Overview

The Odoo POS ↔ NetSuite Integration project establishes a bidirectional, configurable enterprise integration solution between Odoo Point of Sale (POS) system and Oracle NetSuite ERP platform. This integration enables automated synchronization of transactional and master data while supporting multiple execution modes to accommodate diverse business requirements.

1.2 Business Objectives

- **Operational Efficiency:** Eliminate manual data entry and reduce operational overhead
- **Data Consistency:** Maintain synchronized master data across both systems
- **Financial Accuracy:** Ensure accurate and timely financial reporting through consolidated invoicing
- **Scalability:** Support multi-location, multi-subsidiary business operations
- **Flexibility:** Enable dynamic configuration changes without code deployment

1.3 Key Features

Dynamic Configuration Management

- NetSuite-controlled business logic via REST API
- Real-time configuration updates to Odoo
- No hardcoded business rules

Consolidated Transaction Processing

- One consolidated invoice per shop per day
- Aggregated line items with intelligent quantity summation
- Automatic end-of-day processing

Flexible Execution Modes

- Real-time sync on transaction confirmation
- Scheduled batch processing (midnight sync)
- Manual on-demand synchronization

Master Data Synchronization

- Hourly product/item catalog updates
- Payment method mappings
- Multi-subsidiary support (NetSuite OneWorld)

Enterprise-Grade Reliability

- Exponential backoff retry mechanism
- Comprehensive audit logging
- Manual retry capabilities for failed transactions
- Queue-based background processing

1.4 Integration Scope

Master Data (NetSuite → Odoo)

- Products/Items (Inventory Items)
- Payment Methods
- Shop/Subsidiary Mappings
- Location and Department Hierarchies

Transactional Data (Odoo → NetSuite)

- Consolidated Sales Orders (one per shop per day)
- Consolidated Invoices (one per shop per day)
- Aggregated Payment Transactions

1.5 Technical Architecture Summary

The solution implements a **Client-Server Architecture Pattern** where:

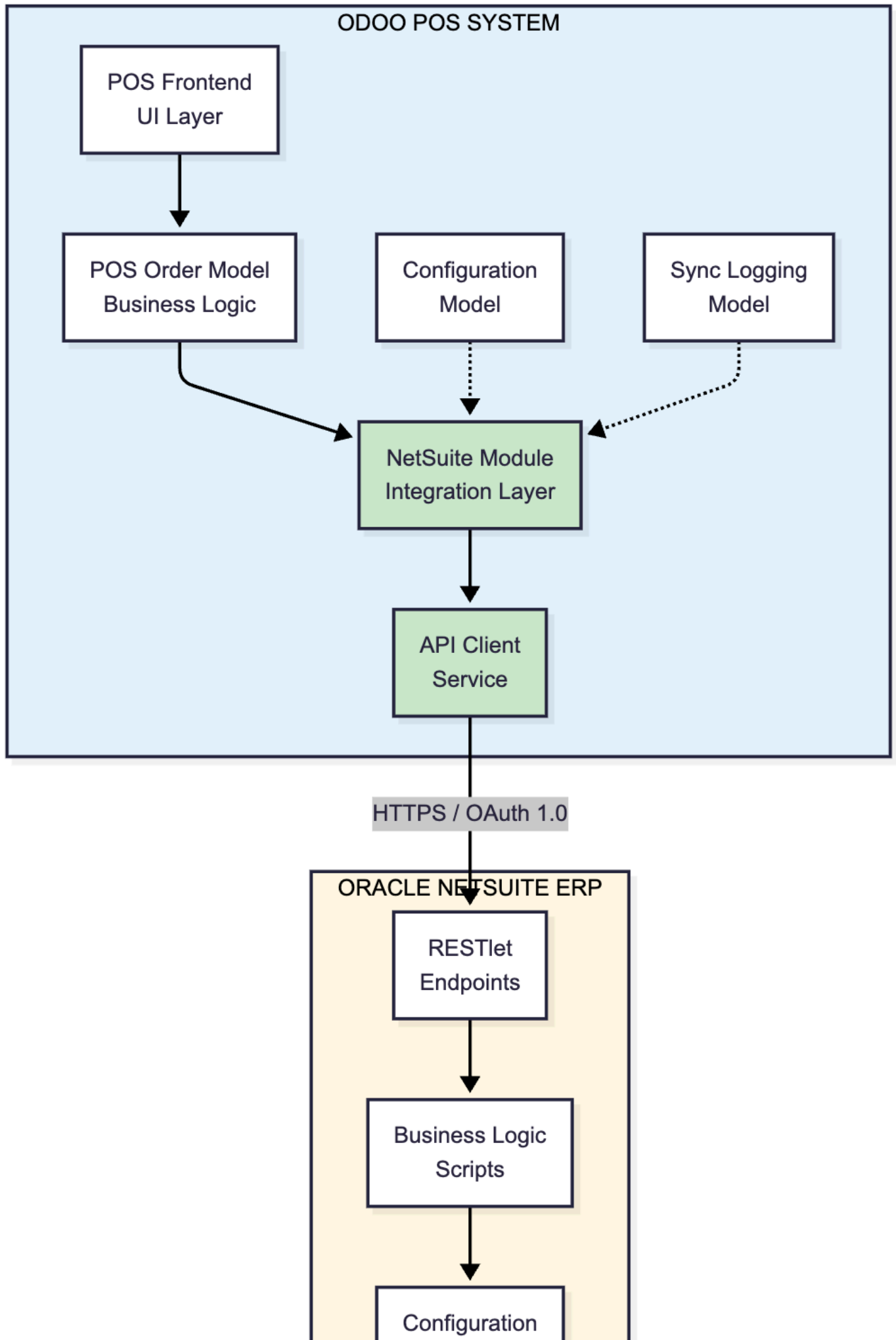
- **Odoo (Client):** "Dumb client" that stores only NetSuite credentials and executes synchronization tasks
- **NetSuite (Server):** "Intelligent server" that controls all business logic, retry policies, and configuration
- **Communication:** RESTful API with OAuth 1.0 authentication

This design ensures:

- Centralized business rule management
- Simplified Odoo module maintenance
- Dynamic reconfiguration without code changes
- Clear separation of concerns

2. System Architecture

2.1 High-Level Architecture





{width=40%}

2.2 Component Architecture

2.2.1 Odoo Components

Component	Type	Responsibility
netsuite.config	Model	Credential storage and configuration management
pos.order (extended)	Model	POS order sync status and NetSuite reference tracking
netsuite.api.client	Service	HTTP communication with NetSuite REST APIs
netsuite.consolidated.sync	Service	Consolidated order/invoice aggregation logic
netsuite.sync.log	Model	Audit trail and sync attempt logging
netsuite.sync.queue	Model	Background job queue management
netsuite.subsidiary.mapping	Model	Shop to NetSuite subsidiary mapping
netsuite.payment.method.mapping	Model	Payment method mapping
Cron Jobs	Scheduler	Automated hourly and end-of-day sync

2.2.2 NetSuite Components

2.3 Technology Stack

Odoo Platform

- **Platform:** Odoo 17.0 Community/Enterprise
- **Language:** Python 3.10+
- **Framework:** Odoo ORM
- **Database:** PostgreSQL 14+
- **API:** Odoo XML-RPC / JSON-RPC
- **HTTP Library:** Python requests library

NetSuite Platform

Infrastructure

- **Containerization:** Docker & Docker Compose
- **Version Control:** Git / GitHub

3. Odoo Implementation Details

3.2 Core Models

3.2.1 NetSuite Configuration Model

Model Name: netsuite.config **Purpose:** Central configuration management and credential storage

Key Fields:

Field Name	Type	Required	Description
name	Char	Yes	Configuration name
active	Boolean	Yes	Enable/disable configuration
api_url	Char	Yes	NetSuite base API URL
account_id	Char	Yes	NetSuite account identifier

Field Name	Type	Required	Description
consumer_key	Char	No	OAuth consumer key
consumer_secret	Char	No	OAuth consumer secret
token_id	Char	No	OAuth token ID
token_secret	Char	No	OAuth token secret
netsuite_config	Text	No	Configuration JSON from NetSuite (read-only)
last_config_fetch	Datetime	No	Last configuration fetch timestamp

Computed Fields (from NetSuite configuration JSON):

Field	Source Path	Description
config_integration_mode	configuration.integration_mode	Current sync mode
config_retry_enabled	configuration.retry_policy.enabled	Retry enabled flag
config_max_retries	configuration.retry_policy.max_retries	Maximum retry attempts
config_consolidate_orders	configuration.consolidation_rules.consolidate_orders_per_shop_per_day	Consolidation flag
config_end_of_day_sync_time	configuration.scheduled_settings.order_sync_time	EOD sync time

Key Methods:

```
@api.model
def get_active_config():
    """Retrieve the active configuration"""

def test_connection(self):
    """Test NetSuite API connectivity"""

def action_sync_products(self):
    """Manually trigger product sync"""
```

3.2.2 POS Order Extension

Model Name: `pos.order` (inherited) **Purpose:** Track NetSuite sync status for POS orders

Additional Fields:

Field Name	Type	Description
netsuite_sync_status	Selection	Sync status (not_synced, queued, synced, failed)
netsuite_id	Char	NetSuite internal ID
netsuite_tran_id	Char	NetSuite transaction number
netsuite_sync_date	Datetime	Last successful sync timestamp
netsuite_error	Text	Error message from last failed sync
netsuite_sync_count	Integer	Number of sync attempts
x_netsuite_invoice_id	Char	Consolidated invoice NetSuite ID
x_netsuite_invoice_sync_date	Datetime	Invoice sync timestamp

Key Methods:

```

def action_sync_to_netsuite(self):
    """
    Manual batch sync to NetSuite
    Creates ONE consolidated invoice per shop per day
    """

def _prepare_netsuite_order_data(self):
    """Prepare order data for NetSuite API"""

def _mark_as_synced(self, netsuite_id, netsuite_tran_id):
    """Update sync status after successful sync"""

def _mark_as_failed(self, error_message):
    """Update sync status after failed sync"""

```

3.2.3 Consolidated Sync Service

Model Name: `netsuite.consolidated.sync` **Type:** Abstract Model (Service) **Purpose:** Aggregate and sync consolidated orders/invoices

Key Methods:

```

@api.model
def sync_consolidated_orders(self, target_date=None, warehouse_ids=None):
    """
    Sync consolidated orders to NetSuite (one per shop per day)

    Args:
        target_date: Date to sync (default: yesterday)
        warehouse_ids: List of warehouse IDs (default: all)

    Returns:
        dict: {success, total_shops, total_orders, synced, failed, errors}
    """

@api.model
def sync_consolidated_invoices(self, target_date=None, warehouse_ids=None):
    """Sync consolidated invoices to NetSuite"""

def _group_orders_by_shop(self, pos_orders):
    """Group orders by warehouse/shop"""

def _aggregate_line_items(self, orders):
    """
    Aggregate order lines by product
    Sums quantities, calculates weighted average prices
    """

def _prepare_consolidated_payload(self, warehouse, orders, aggregated_lines, target_date):
    """Prepare consolidated invoice payload for NetSuite API"""

```

3.2.4 API Client Service

Model Name: `netsuite.api.client` **Type:** Abstract Model (Service) **Purpose:** Handle all HTTP communication with NetSuite

Key Methods:

```

@api.model
def _get_headers(self, config):
    """Generate HTTP headers with OAuth signature"""

@api.model
def _make_request(self, config, endpoint, method='POST', data=None):
    """
    Make HTTP request to NetSuite

    Returns:
        tuple: (success, response_data, error_message, status_code, execution_time)
    """

```

```
#####

@api.model
def fetch_config(self, config):
    """Fetch configuration from NetSuite"""

@api.model
def test_connection(self, config):
    """Test NetSuite connectivity"""

@api.model
def sync_product(self, config, product_data):
    """Sync individual product to NetSuite"""

@api.model
def create_consolidated_order(self, config, order_payload):
    """Create consolidated order in NetSuite"""

@api.model
def create_consolidated_invoice(self, config, invoice_payload):
    """Create consolidated invoice in NetSuite"""
```

Request Flow:

- 1. Retrieve active configuration
- 2. Generate OAuth headers (if credentials configured)
- 3. Construct full URL from base URL + endpoint
- 4. Execute HTTP request with timeout settings
- 5. Parse JSON response
- 6. Log request/response for audit
- 7. Return standardized response tuple

3.2.5 Sync Logging Model

Model Name: `netsuite.sync.log` **Purpose:** Comprehensive audit trail for all sync operations

Fields:

Field	Type	Description
<code>name</code>	Char	Log entry identifier
<code>sync_type</code>	Selection	Type: order, invoice, product, config
<code>sync_direction</code>	Selection	Direction: odoo_to_netsuite, netsuite_to_odoo
<code>status</code>	Selection	Status: success, failed, pending
<code>request_payload</code>	Text	JSON request payload
<code>response_payload</code>	Text	JSON response payload
<code>error_message</code>	Text	Error details if failed
<code>execution_time_ms</code>	Integer	Request execution time
<code>http_status_code</code>	Integer	HTTP response status
<code>pos_order_ids</code>	Many2many	Related POS orders
<code>sync_date</code>	Datetime	Sync attempt timestamp
<code>retry_count</code>	Integer	Number of retries

3.3 Mapping Models

3.3.1 Subsidiary Mapping

Model Name: `netsuite.subsidiary.mapping` **Purpose:** Map Odoo shops/warehouses to NetSuite subsidiaries

Fields:

Field	Type	Description
<code>name</code>	Char	Mapping name
<code>odoo_warehouse_id</code>	Many2one	Odoo warehouse/shop
<code>netsuite_subsidary_id</code>	Char	NetSuite subsidiary internal ID
<code>netsuite_subsidary_name</code>	Char	NetSuite subsidiary name
<code>netsuite_department_id</code>	Char	NetSuite department ID (optional)
<code>netsuite_location_id</code>	Char	NetSuite location ID (optional)
<code>active</code>	Boolean	Enable/disable mapping

3.3.2 Payment Method Mapping

Model Name: `netsuite.payment.method.mapping` **Purpose:** Map Odoo payment methods to NetSuite payment methods

Fields:

Field	Type	Description
<code>name</code>	Char	Mapping name
<code>odoo_payment_method_id</code>	Many2one	Odoo payment method
<code>netsuite_payment_method_id</code>	Char	NetSuite payment method ID
<code>netsuite_payment_method_name</code>	Char	NetSuite payment method name
<code>active</code>	Boolean	Enable/disable mapping

3.4 Automated Schedulers (Cron Jobs)

3.4.1 Hourly Product Sync

Cron Name: `NetSuite: Sync Products Hourly` **Schedule:** Every hour at :00 **Model:** `netsuite.product.sync` **Method:** `cron_sync_products_from_netsuite`

Logic:

1. Fetch active NetSuite configuration
2. Call NetSuite REST API: `/services/rest/record/v1/inventoryItem`
3. Parse response and map fields to Odoo products
4. Create or update products in Odoo
5. Log sync results

Field Mapping (NetSuite → Odoo):

NetSuite Field	Odoo Field	Notes
<code>itemid</code>	<code>default_code</code>	Product internal reference
<code>displayname</code>	<code>name</code>	Product name
<code>salesdescription</code>	<code>description_sale</code>	Sales description
<code>cost</code>	<code>standard_price</code>	Product cost
<code>price</code>	<code>list_price</code>	Sale price
<code>quantityavailable</code>	<code>qty_available</code>	Stock quantity
<code>isinactive</code>	<code>active</code>	Active status (inverted)

3.4.2 End of Day Order Sync

Cron Name: `NetSuite: Sync Orders End of Day` **Schedule:** Daily at 00:00 (midnight) **Model:** `netsuite consolidated.sync` **Method:** `cron_sync_end_of_day_orders`

Logic:

1. Calculate target date (previous day)

2. Fetch all paid POS orders from previous day
3. Group orders by warehouse/shop
4. For each shop:
 - Aggregate all order lines by product
 - Calculate total quantities and weighted average prices
 - Generate consolidated invoice payload
 - Send to NetSuite API
 - Mark orders as synced
5. Log all results

Payload Structure:

```
{
  "type": "consolidated_invoice",
  "subsidiary_id": "123",
  "transaction_date": "2026-05-12",
  "shop_name": "Downtown Store",
  "total_orders": 45,
  "line_items": [
    {
      "item_id": "PROD-001",
      "quantity": 125,
      "rate": 19.99,
      "amount": 2498.75
    }
  ],
  "payments": [
    {
      "payment_method": "Credit Card",
      "amount": 15234.50
    }
  ]
}
```

3.5 Manual Sync Operations

3.5.1 Batch Order Sync

Location: Point of Sale → Orders (list view) **Action:** "Sync to NetSuite" (multi-select action)

Business Rules:

- Cannot sync today's orders (must wait for EOD)
- Cannot sync already synced orders
- Groups orders by (shop, date) automatically
- Creates one consolidated invoice per group

Wizard: [netsuite.manual.sync.wizard](#)

3.5.2 Test Connection

Location: NetSuite → Configuration **Button:** "Test Connection"

Validates:

- API URL reachability
- OAuth credentials (if configured)
- NetSuite account accessibility
- Configuration endpoint availability

3.6 User Interface

Configuration Views: Located in [NetSuite](#) → [Configuration](#) menu with forms for API credentials and sync settings.

POS Order Extensions: Additional sync status column, NetSuite ID fields, and batch sync actions in Point of Sale → Orders list view.

Sync Logs: Comprehensive logging interface under NetSuite → Sync Logs with filters for type, status, and date range.

Mapping Views: Subsidiary and payment method mapping screens under NetSuite → Mappings menu.

4. NetSuite Implementation Details

[TO BE DOCUMENTED BY NETSUITE DEVELOPMENT TEAM]

5. API Specifications

5.1 Odoo REST API Endpoints

5.1.1 Configuration Update Endpoint

Endpoint: `/api/netsuite/config/update` **Method:** POST **Authentication:** Odoo Database Authentication **Purpose:** Receive configuration updates from NetSuite

Headers:

```
Content-Type: application/json
db: {database_name}
login: {user_login}
password: {user_password}
```

Request Body:

```
{
  "configuration": {
    "integration_mode": "scheduled",
    "realtime_settings": {
      "enabled": false,
      "sync_on_order_confirmed": false
    },
    "scheduled_settings": {
      "enabled": true,
      "order_sync_time": "00:00",
      "invoice_sync_time": "00:00",
      "product_sync_frequency": "hourly",
      "product_sync_hour_interval": 1
    },
    "retry_policy": {
      "enabled": true,
      "max_retries": 3,
      "initial_delay_minutes": 5,
      "use_exponential_backoff": true,
      "backoff_multiplier": 2
    },
    "batch_processing": {
      "order_batch_size": 100,
      "invoice_batch_size": 100,
      "product_batch_size": 50
    },
    "notification": {
      "send_email_on_failure": true,
      "send_email_on_success": false,
      "notification_recipients": ["admin@example.com"]
    },
    "consolidation_rules": {
      "consolidate_orders_per_shop_per_day": true,
      "consolidate_invoices_per_shop_per_day": true,
      "aggregate_line_items": true,
      "group_by_product": true
    }
  },
  "metadata": {
    "config_version": "1.0",
    "last_updated_by": "NetSuite System",
    "last_updated_at": "2026-05-13T10:30:00Z",
    "netsuite_environment": "production"
  }
}
```

```
}  
}
```

Success Response:

```
{  
  "success": true,  
  "message": "Configuration updated successfully",  
  "config_id": 1,  
  "applied_at": "2026-05-13T10:30:15Z"  
}
```

Error Response:

```
{  
  "success": false,  
  "error": "Invalid configuration JSON structure",  
  "details": "Missing required field: configuration.integration_mode"  
}
```

5.2 NetSuite REST API Endpoints

Expected Request Payload (from Odoo):

```
{  
  "type": "consolidated_invoice",  
  "subsidiary_id": "123",  
  "transaction_date": "2026-05-12",  
  "shop_info": {  
    "odoo_warehouse_id": 1,  
    "shop_name": "Downtown Store",  
    "shop_code": "DS-001"  
  },  
  "summary": {  
    "total_orders": 45,  
    "total_line_items": 8,  
    "total_amount": 15234.50,  
    "total_tax": 1218.76  
  },  
  "line_items": [  
    {  
      "item_id": "PROD-001",  
      "item_name": "Product Name",  
      "quantity": 125.0,  
      "rate": 19.99,  
      "amount": 2498.75,  
      "tax_code": "TAX-STD"  
    }  
  ],  
  "payments": [  
    {  
      "payment_method_id": "PM-CC",  
      "payment_method_name": "Credit Card",  
      "amount": 10000.00,  
      "transaction_count": 28  
    },  
    {  
      "payment_method_id": "PM-CASH",  
      "payment_method_name": "Cash",  
      "amount": 5234.50,  
      "transaction_count": 17  
    }  
  ],  
  "metadata": {  
    "odoo_order_ids": [1001, 1002, 1003],  
    "sync_timestamp": "2026-05-13T00:05:00Z",  
  }  
}
```

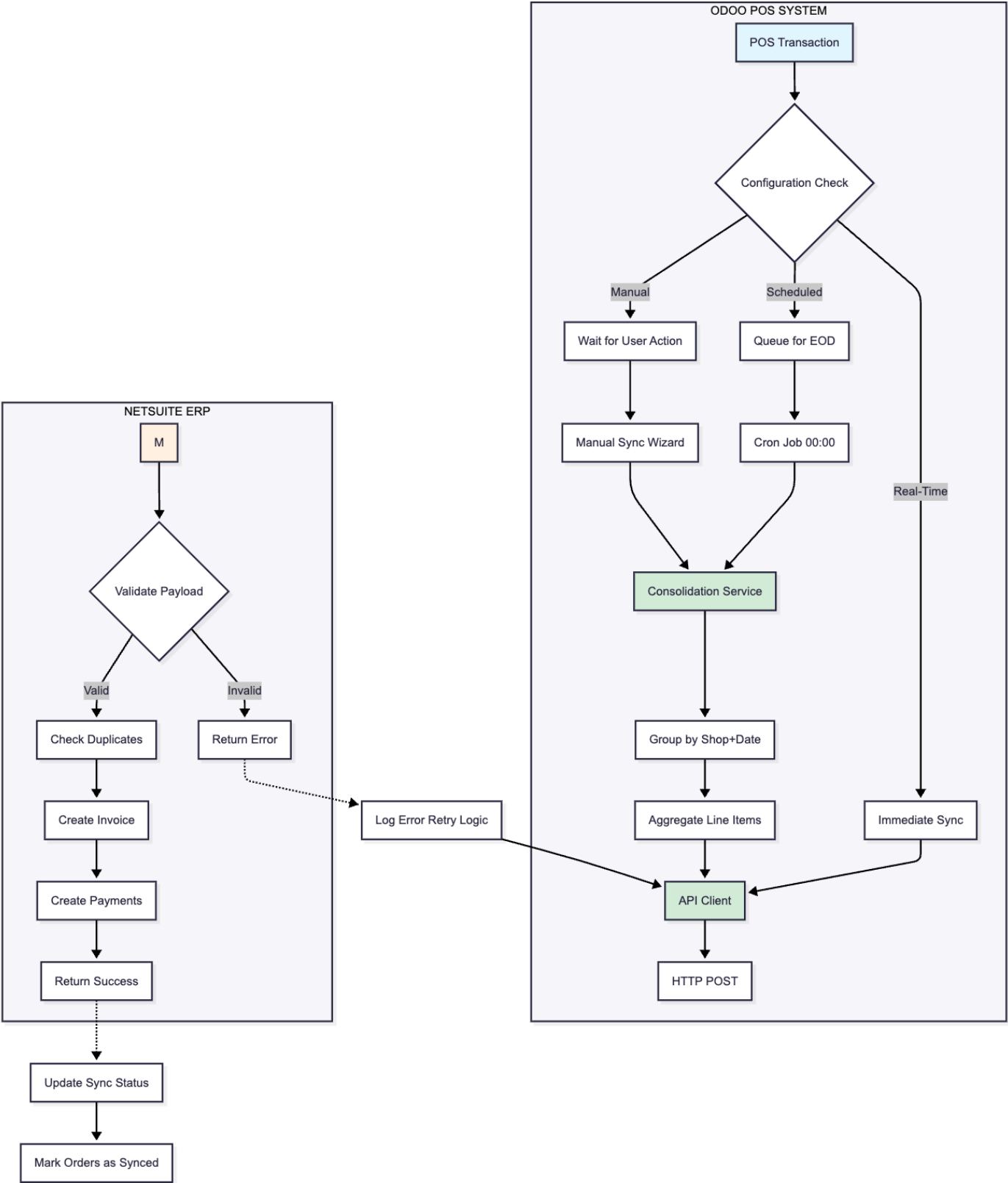
```
"odoo_user_id": 2,  
"odoo_user_name": "System Scheduler"  
}  
}
```

Expected Response (from NetSuite):

```
{  
  "success": true,  
  "netsuite_invoice_id": "12345",  
  "netsuite_transaction_id": "INV-2026-05-12-001",  
  "created_at": "2026-05-13T00:05:10Z",  
  "message": "Consolidated invoice created successfully"  
}
```

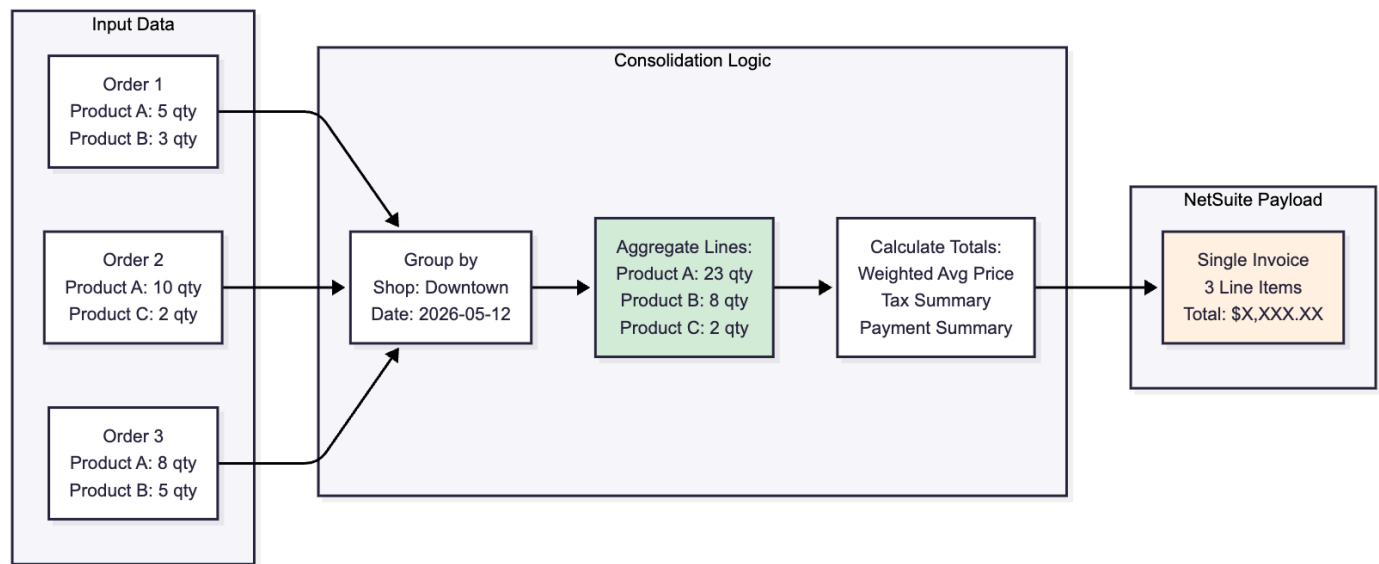
6. Data Flow and Process Logic

6.1 System Integration Flow



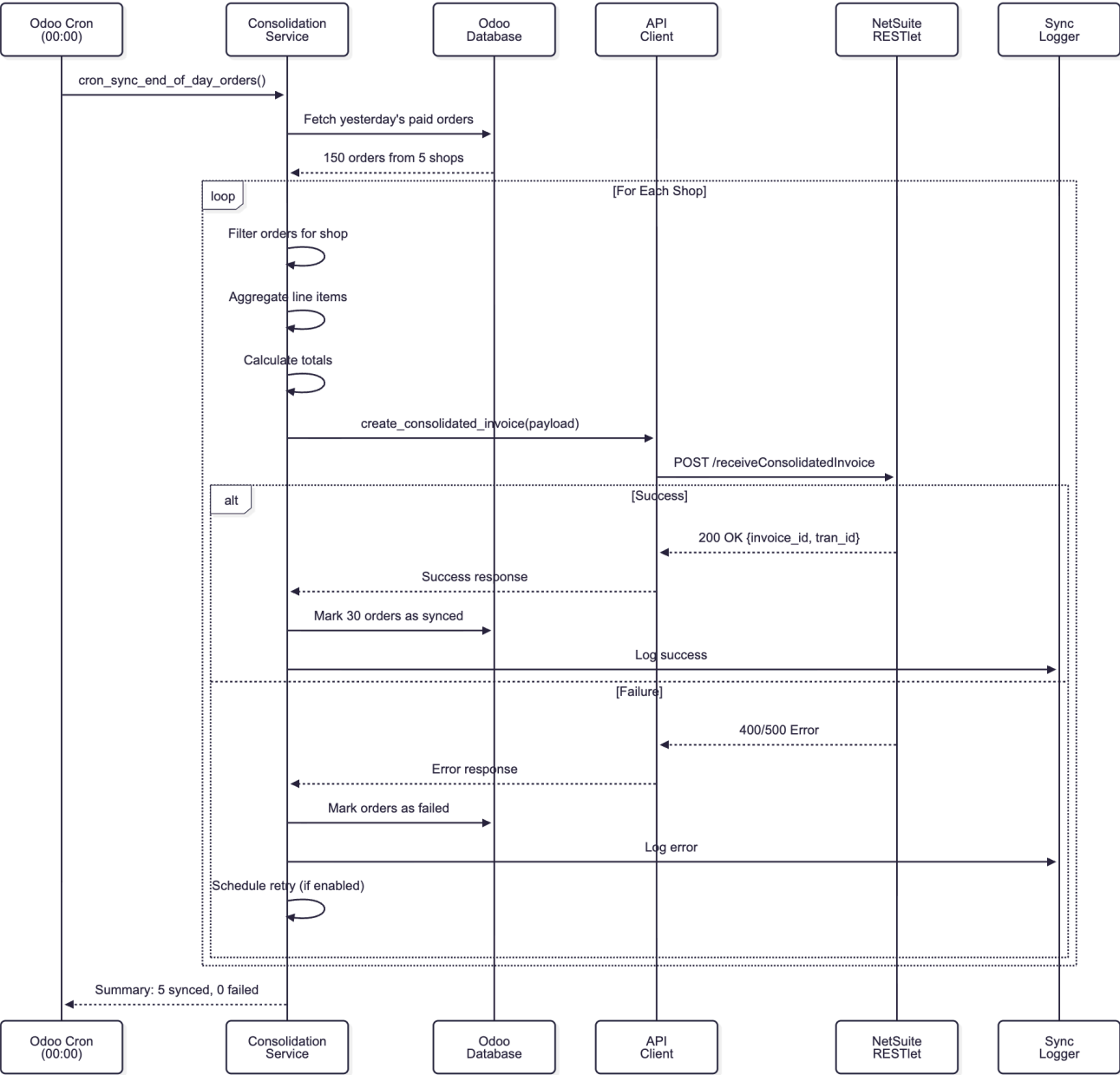
{width=75%}

6.2 Consolidated Invoice Generation Flow



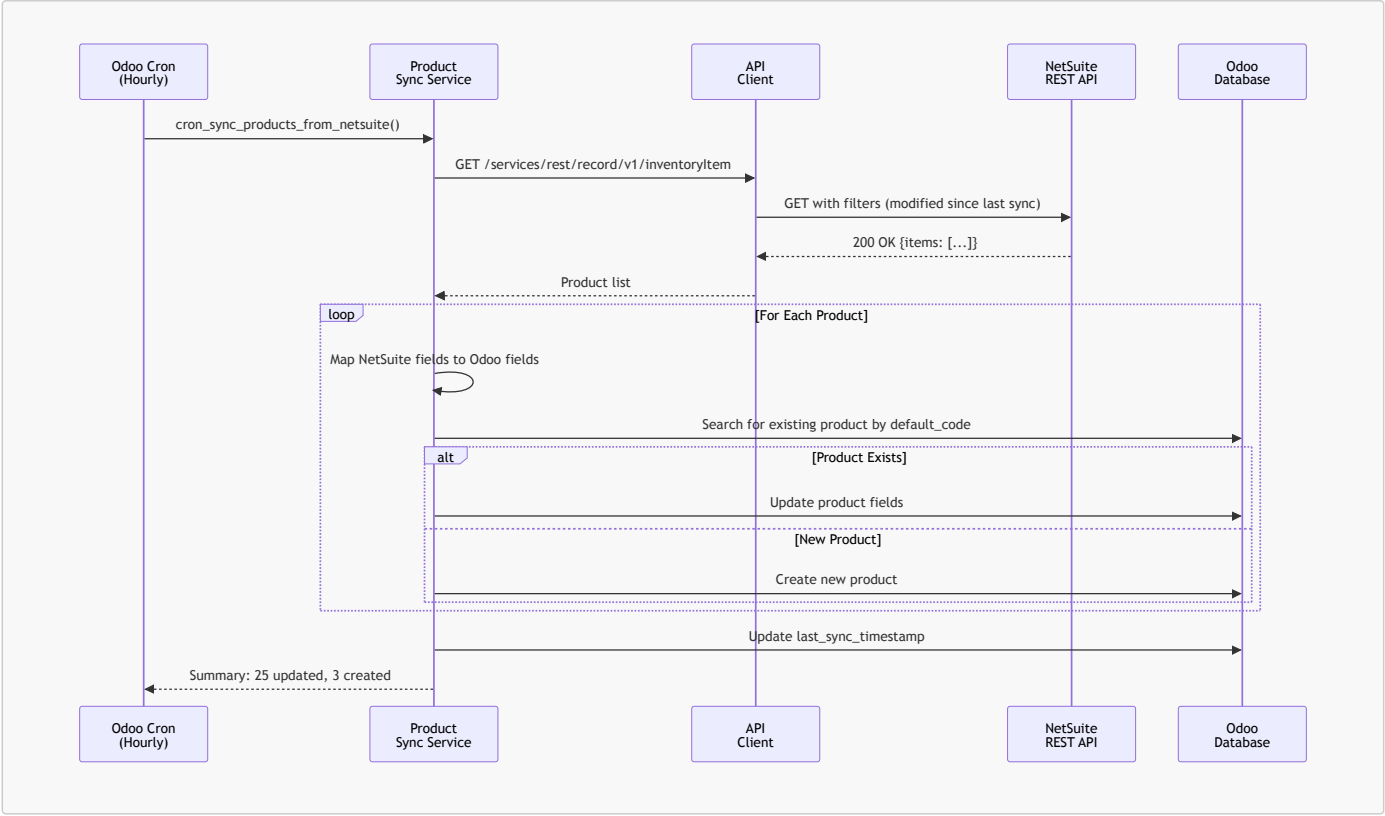
{width=75%}

6.3 End of Day Sync Process

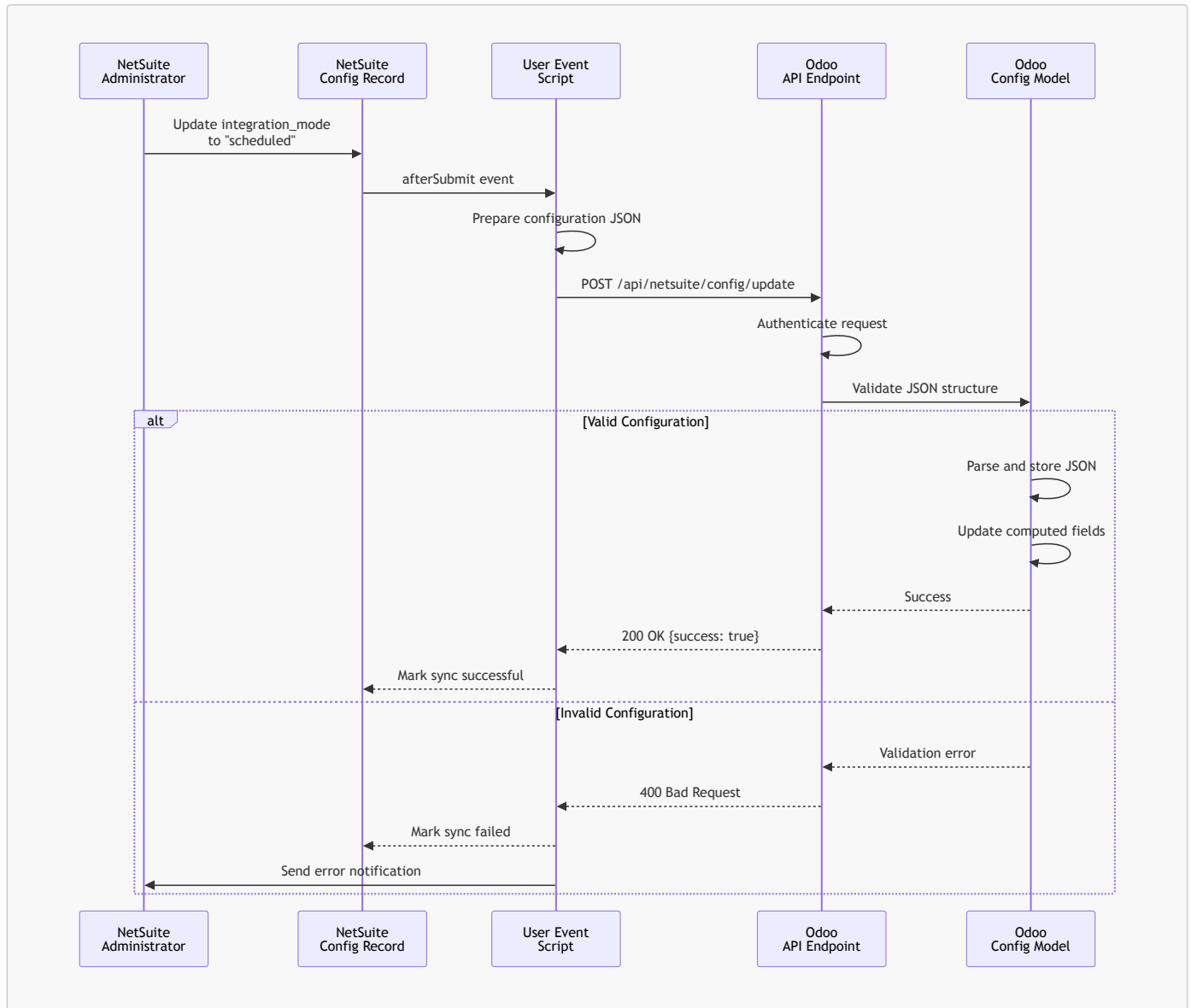


{width=75%}

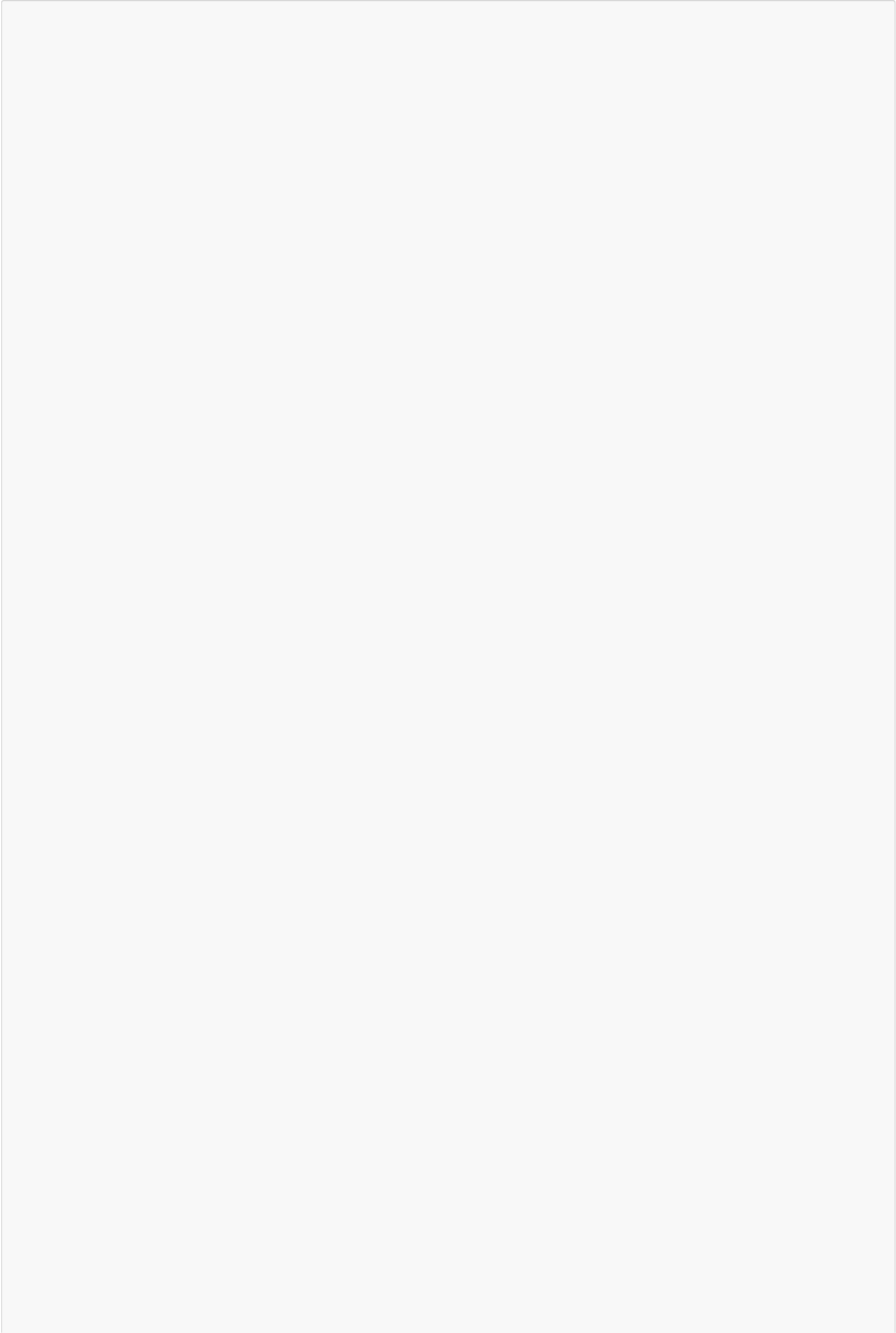
6.4 Product Sync Flow (Hourly)

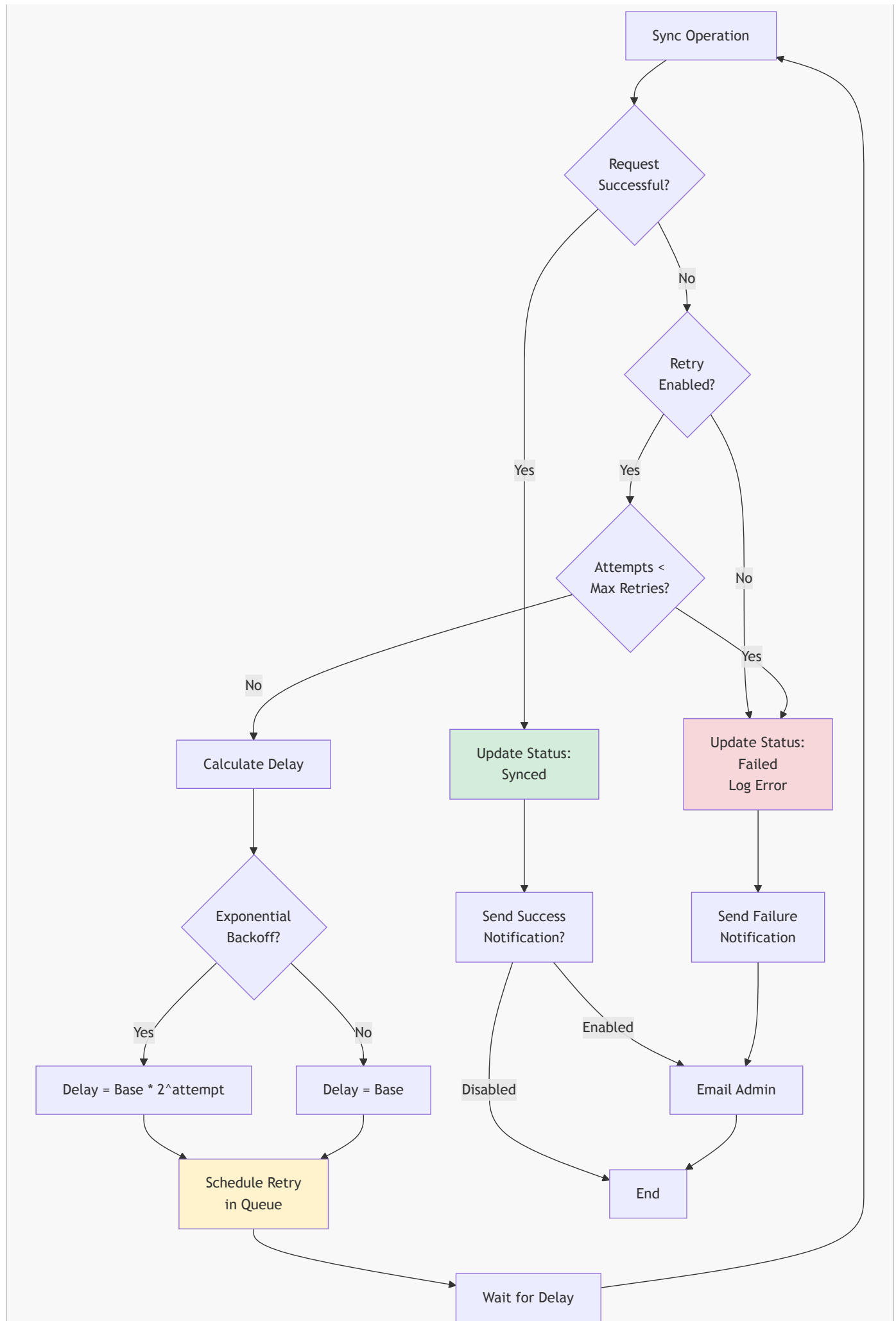


6.5 Configuration Update Flow



6.6 Error Handling and Retry Flow





7. Configuration Management

7.1 Configuration Schema

The complete configuration JSON schema is defined in `CONFIGURATION_SCHEMA.md`. Key sections:

7.1.1 Integration Modes

Mode	Description	Use Case
realtime	Sync immediately on transaction confirmation	High-frequency, time-sensitive operations
scheduled	Sync at configured times (EOD batch)	Normal operations, reduced API load
manual	Sync only when manually triggered	Testing, controlled deployments

7.1.2 Retry Policy Configuration

```
{
  "retry_policy": {
    "enabled": true,
    "max_retries": 3,
    "initial_delay_minutes": 5,
    "use_exponential_backoff": true,
    "backoff_multiplier": 2
  }
}
```

Retry Schedule Example:

- Attempt 1: Immediate
- Attempt 2: After 5 minutes
- Attempt 3: After 10 minutes (5 × 2)
- Attempt 4: After 20 minutes (10 × 2)

7.1.3 Consolidation Rules

```
{
  "consolidation_rules": {
    "consolidate_orders_per_shop_per_day": true,
    "consolidate_invoices_per_shop_per_day": true,
    "aggregate_line_items": true,
    "group_by_product": true
  }
}
```

7.2 Configuration Synchronization

NetSuite → Odoo:

- NetSuite pushes configuration changes via POST API
- Odoo validates and stores JSON
- Computed fields automatically update
- No Odoo restart required

Update Mechanism:

- Automatic: Via `/api/netsuite/config/update` endpoint when NetSuite configuration changes

7.3 Configuration Validation

Odoo-Side Validation:

```
def _validate_config_json(self, config_json):
    """Validate NetSuite configuration JSON structure"""
    required_keys = ['configuration', 'metadata']
    if not all(key in config_json for key in required_keys):
        raise ValidationError(_('Invalid configuration structure'))

    config = config_json['configuration']
    if 'integration_mode' not in config:
        raise ValidationError(_('Missing integration_mode'))

    if config['integration_mode'] not in ['realtime', 'scheduled', 'manual']:
        raise ValidationError(_('Invalid integration_mode value'))

    # Additional validations...
```

8. Security and Authentication

8.1 Odoo API Security

Authentication Methods:

- Database name + Login + Password (API keys recommended)
- Session-based authentication for UI access

Access Control:

- Group: **NetSuite Integration User** (read access to logs, manual sync)
- Group: **NetSuite Integration Manager** (full configuration access)

8.2 Data Privacy

- OAuth credentials stored in encrypted database fields
- API responses logged for audit
- Configuration options for logging sensitive data

9. Error Handling and Logging

9.1 Error Categories

Category	Severity	Handling Strategy
Network Errors	High	Automatic retry with exponential backoff
Authentication Errors	Critical	Manual intervention required, notify admin
Validation Errors	Medium	Log and skip, notify for manual review
Business Logic Errors	Medium	Retry with same payload
Configuration Errors	Critical	Halt sync, notify admin immediately

9.2 Logging Strategy

9.2.1 Sync Log Fields

Every sync operation creates a log entry with:

- Request timestamp
- Request payload (configurable)
- Response payload (configurable)
- HTTP status code
- Execution time (milliseconds)
- Error message (if failed)
- Retry count
- Related Odoo records

9.2.2 Log Retention

Configurable via:

```
{
  "logging": {
    "log_retention_days": 90
  }
}
```

Automated cleanup cron job removes logs older than retention period.

9.2.3 Error Notifications

Email Notifications:

- Configurable recipients list
- Template: Sync failure summary
- Trigger conditions:
 - Max retries exhausted
 - Authentication failure
 - Configuration fetch failure

Sample Notification:

```
Subject: NetSuite Sync Failure - Immediate Attention Required

Dear Administrator,

A NetSuite synchronization operation has failed after exhausting all retry attempts.

Details:
- Sync Type: Consolidated Invoice
- Shop: Downtown Store
- Date: 2026-05-12
- Total Orders: 45
- Error: Connection timeout after 60 seconds
- Attempts: 4/3
- Last Attempt: 2026-05-13 01:35:00 UTC

Action Required:
Please review the sync logs and manually retry if necessary.

View Logs: [Link to Odoo Sync Logs]
```

9.3 Monitoring and Alerts

Key Metrics to Monitor:

- Sync success rate (daily/weekly)
- Average execution time
- Failed sync count
- Retry frequency
- Configuration fetch failures

Monitoring Tools:

- Odoo built-in logging